



Measuring Quality

Guide for the DPI-LS Project

Digital Performance Index — Quality Dimension (Q)

Accuracy, Consistency & Hallucination Tracking

Framework: Digital Performance Index (DPI-LS)

Dimension: Quality (Q) — Weight: 20%

Version: 2.5 (Extended Enterprise Edition)

Status: Architectural Reference

DPI-LS Architecture Team

Document Developed by **Malkapurapu Sivamanikanta**

Contents

1	Executive Summary & Introduction	2
2	The SME & LLM-Judge Analogy	2
3	The Mathematics of Quality	3
3.1	The Quality Equation	3
3.2	Deep Mathematical Breakdown of Variables	3
3.2.1	The Accuracy Derivation (A)	3
3.2.2	The Consistency Derivation (C)	5
3.2.3	The Hallucination Derivation (H)	6
3.2.4	The Final Composite Calculation	6
3.3	The Null Fallback Mechanism	7
4	Quality Observability Architecture	8
4.1	RAG Observability & Context Entailment	8
4.2	The Quality Score Calculation Flow	8
4.3	The SME Fallback Workflow	9
5	Business Scenarios & Root Cause Analysis	10
6	Telemetry & Integrations	12
7	Quality Telemetry Contract	13
8	Quality Thresholds & Alerting	13
9	Conclusion & Next Steps	14
9.1	Next Steps Roadmap	14

1 Executive Summary & Introduction

NOTE

The **Digital Performance Index (DPI-LS)** framework quantifies AI agent performance across 7 distinct dimensions. Within this framework, the **Quality (Q)** dimension evaluates the correctness, consistency, and reliability of the agent's output.

Measuring the quality of non-deterministic LLM output is one of the most challenging aspects of modern FinOps and AI Governance. Unlike simple latency or unit costs, "Quality" is subjective. Does the model hallucinate facts? Is it consistent when asked the same question twice?

This document outlines the architecture for the **Quality** dimension, explaining its mathematically rigorous evaluation framework (70% Accuracy, 20% Consistency, 10% Hallucination Rate) and how the DPI-LS Engine seamlessly falls back to Human-in-the-Loop Subject Matter Experts (SMEs) when automated "LLM-as-a-Judge" metrics are insufficient.

2 The SME & LLM-Judge Analogy

The Automated Teacher Analogy

 **The Test and the Teacher:** Imagine a classroom where students (the AI Agents) are constantly turning in essays (outputs). To grade 10,000 essays a minute, the school hires an automated grading robot (LLM-as-a-Judge). The robot checks for three things: 1. **Accuracy (70%):** Did the student get the right answer? 2. **Consistency (20%):** Does the student contradict themselves? 3. **Hallucination (10%):** Did the student invent a fake historical event?

 **The Principal (Human SME):** However, if the student writes a highly complex, nuanced essay about advanced quantum mechanics, the robot might get confused and return a None score. When this happens, the robot hands the essay directly to the Principal (a human Subject Matter Expert). The Principal manually grades the essay, ensuring that complex outputs are always judged accurately rather than being arbitrarily failed by the machine.

3 The Mathematics of Quality

The DPI-LS Engine treats the Quality score as a composite metric of three sub-metrics. All inputs must be normalized floats between 0.0 and 1.0.

3.1 The Quality Equation

The total Quality (Q) is calculated as:

DPI-LS Quality Formula

$$Q = (0.7 \times \text{Accuracy}) + (0.2 \times \text{Consistency}) + (0.1 \times (1 - \text{Hallucination Rate}))$$

Variable Breakdown:

- **Accuracy** ($w = 0.7$): The semantic correctness of the output compared to ground truth (usually measured via Ragas or TruLens).
- **Consistency** ($w = 0.2$): The stability of the output over multiple identical queries (measured via Self-Consistency checks).
- **Hallucination Rate** ($w = 0.1$): The percentage of unsupported or fabricated claims. Note that the formula uses $(1 - \text{Hallucination})$ so that a higher score is better.

3.2 Deep Mathematical Breakdown of Variables

The weights (0.7, 0.2, 0.1) are carefully calibrated. An agent that is perfectly consistent but entirely hallucinatory is completely useless, whereas an agent that is accurate but occasionally inconsistent requires only minor prompt tuning.

To understand why the DPI-LS engine uses these specific variables, we must explore the mathematical derivation of each underlying float.

3.2.1 The Accuracy Derivation (A)

Accuracy (A) is the semantic correctness of the output compared to ground truth. In traditional deterministic software, accuracy is boolean (1 or 0). In generative AI, accuracy is measured continuously via **Cosine Similarity** in high-dimensional vector spaces.

When an LLM Judge (e.g., TruLens) evaluates an agent, it uses an embedding model (like `text-embedding-3-small`) to map the agent's output and the ground-truth answer into a 1,536-dimensional vector space.

Given two embedding vectors, A_{out} (the agent's output) and B_{gt} (the ground-truth document), the accuracy is derived from the dot product of the vectors divided by the product of their magnitudes:

Cosine Similarity Equation

$$\text{Accuracy} = \cos(\theta) = \frac{A_{out} \cdot B_{gt}}{\|A_{out}\| \|B_{gt}\|} = \frac{\sum_{i=1}^n A_{out,i} B_{gt,i}}{\sqrt{\sum_{i=1}^n A_{out,i}^2} \sqrt{\sum_{i=1}^n B_{gt,i}^2}}$$

Step-by-Step Example Calculation: Assume for simplicity a 3-dimensional embedding space:

- Ground Truth $B_{gt} = [1.0, 0.5, 0.2]$
- Agent Output $A_{out} = [0.9, 0.6, 0.1]$

1. Calculate the Dot Product:

$$A_{out} \cdot B_{gt} = (1.0 \times 0.9) + (0.5 \times 0.6) + (0.2 \times 0.1) = 0.9 + 0.3 + 0.02 = 1.22$$

2. Calculate the Magnitudes:

$$\|B_{gt}\| = \sqrt{1.0^2 + 0.5^2 + 0.2^2} = \sqrt{1.0 + 0.25 + 0.04} = \sqrt{1.29} \approx 1.135$$

$$\|A_{out}\| = \sqrt{0.9^2 + 0.6^2 + 0.1^2} = \sqrt{0.81 + 0.36 + 0.01} = \sqrt{1.18} \approx 1.086$$

3. Calculate the Final Accuracy Float:

$$\text{Accuracy} = \frac{1.22}{1.135 \times 1.086} = \frac{1.22}{1.232} \approx 0.99$$

The resulting float 0.99 indicates near-perfect semantic alignment, even if the exact wording of the agent's output differed from the ground truth.

3.2.2 The Consistency Derivation (C)

Consistency measures the stability of the output over multiple identical queries. Because LLMs sample from a probability distribution over tokens, setting a high temperature introduces stochasticity.

Consistency is calculated using **Self-Consistency** sampling over k iterations and evaluating the **Shannon Entropy** of the resulting response clusters.

The Discrete Probability Approach: If we query the agent $k = 5$ times with the exact same prompt, we map the responses into semantic clusters $V = \{v_1, v_2, \dots, v_m\}$.

Let $p(v_i)$ be the probability of the agent generating a response belonging to cluster v_i :

$$p(v_i) = \frac{\text{count}(v_i)}{k}$$

The Consistency score is calculated using the Normalized Shannon Entropy (H_e) of the distribution:

Normalized Shannon Entropy Equation

$$H_e = - \sum_{i=1}^m p(v_i) \log_2(p(v_i))$$

$$\text{Consistency} = 1 - \left(\frac{H_e}{\log_2(k)} \right)$$

Numerical Example: Assume $k = 5$. The agent generates 3 correct answers (Cluster 1), 1 incorrect answer (Cluster 2), and 1 refusal (Cluster 3).

- $p(v_1) = 3/5 = 0.6$
- $p(v_2) = 1/5 = 0.2$
- $p(v_3) = 1/5 = 0.2$

1. Calculate the raw Entropy:

$$H_e = -[(0.6 \times \log_2(0.6)) + (0.2 \times \log_2(0.2)) + (0.2 \times \log_2(0.2))]$$

$$H_e \approx -[(0.6 \times -0.737) + (0.2 \times -2.32) + (0.2 \times -2.32)]$$

$$H_e \approx -[-0.442 - 0.464 - 0.464] = 1.37$$

2. Normalize and Invert to find Consistency: The maximum possible entropy for $k = 5$ is $\log_2(5) \approx 2.32$.

$$\text{Consistency} = 1 - \left(\frac{1.37}{2.32} \right) = 1 - 0.59 = \mathbf{0.41}$$

A Consistency score of 0.41 severely penalizes the agent for returning 3 distinct types of answers across 5 runs.

3.2.3 The Hallucination Derivation (H)

Hallucination Rate (H) measures the percentage of fabricated claims. It relies on **Natural Language Inference (NLI)** and **Context Entailment**. For every sentence generated by the agent, a cross-encoder model evaluates the logical relationship between the premise (the retrieved RAG context) and the hypothesis (the agent's sentence).

The cross-encoder outputs raw logits for three classes: Entailment, Neutral, and Contradiction. These logits are mapped to probabilities using the **Softmax function**.

NLI Softmax Equation

$$P(y_i) = \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j \in \{E, N, C\}} e^{z_j}}$$

A sentence s_i is flagged as a hallucination if $P(\text{Contradiction}) > P(\text{Entailment})$.

The aggregate Hallucination Rate for the entire response of N sentences is:

$$H_{rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(P(\text{Contradiction}_{s_i}) > P(\text{Entailment}_{s_i}))$$

Where $\mathbb{I}$ is the indicator function returning 1 if hallucinated and 0 if entailed.

Numerical Matrix Example: An agent generates a 3-sentence response based on a RAG document. The LLM Judge cross-encoder produces the following logit matrices for each sentence:

Sentence	Logit E	Logit N	Logit C	Softmax Output
Sentence 1	4.2	0.1	-2.0	$P(E) = 0.98$ (Entailed)
Sentence 2	-1.5	1.2	3.8	$P(C) = 0.93$ (Hallucinated)
Sentence 3	3.5	0.5	-1.0	$P(E) = 0.95$ (Entailed)

Since 1 out of the 3 sentences triggered a Contradiction probability greater than Entailment, the Hallucination Rate is:

$$H_{rate} = \frac{1}{3} = 0.333$$

3.2.4 The Final Composite Calculation

Now we bring all three highly rigorous mathematical derivations back to the engine's primary Quality equation:

$$Q = (0.7 \times A) + (0.2 \times C) + (0.1 \times (1 - H_{rate}))$$

Using the floats derived from the previous step-by-step examples:

- $A = 0.99$ (Cosine Similarity)
- $C = 0.41$ (Shannon Entropy / Self-Consistency)

- $H_{rate} = 0.333$ (NLI Softmax)

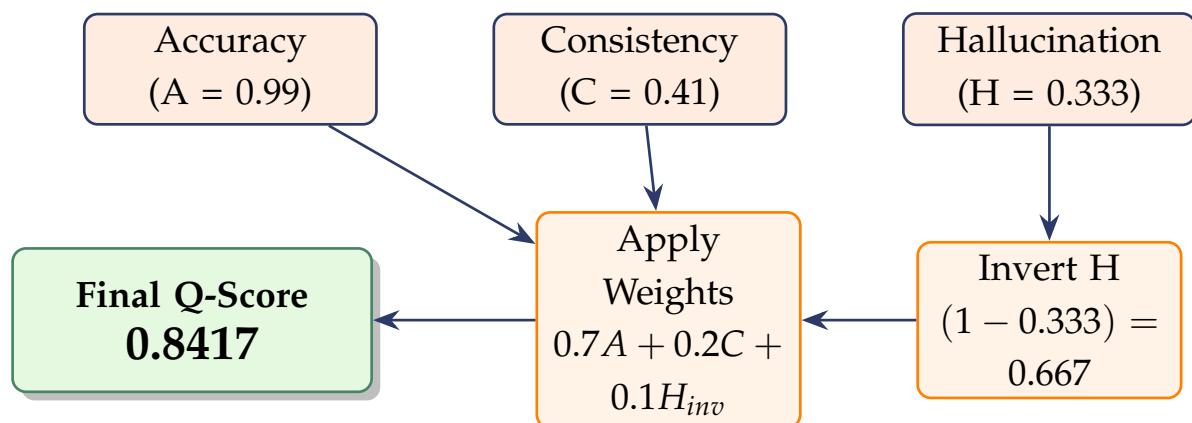
$$Q = (0.7 \times 0.99) + (0.2 \times 0.41) + (0.1 \times (1 - 0.333))$$

$$Q = 0.693 + 0.082 + (0.1 \times 0.667)$$

$$Q = 0.693 + 0.082 + 0.0667$$

$$Q = \mathbf{0.8417}$$

Because the final Q score is 0.8417, it falls just short of the 0.85 "Excellent" threshold defined in Section 8, triggering an async human review flag.



3.3 The Null Fallback Mechanism

The DPI-LS Engine is strictly designed to handle incomplete observability gracefully. If a telemetry payload omits the quality block (returning None):

⚠ IMPORTANT

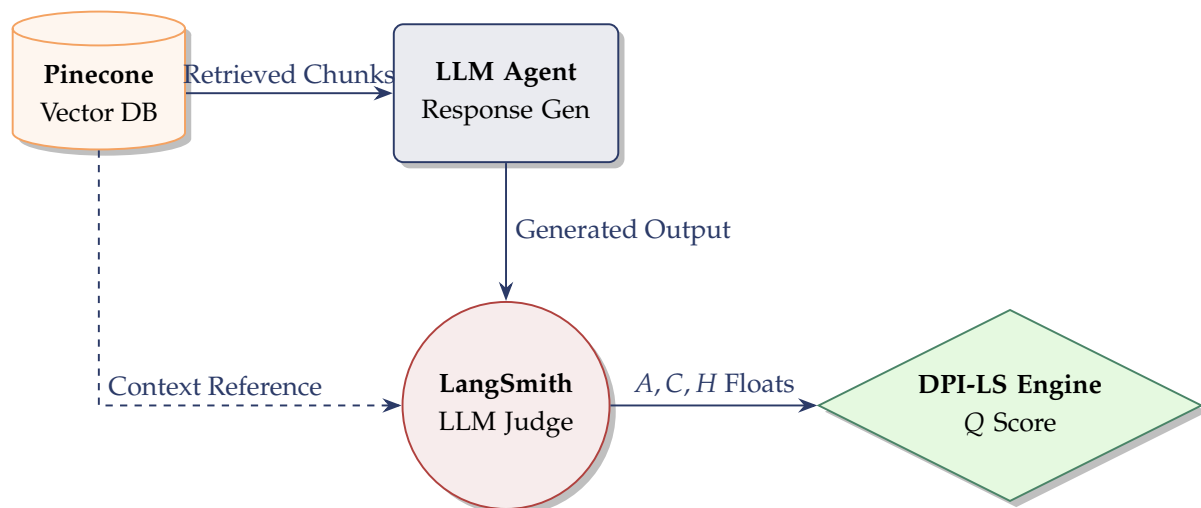
The Engine does **not** treat a None Quality score as a zero. A zero would artificially plummet the agent's rating. Instead, None signals the engine to redistribute the dimension weights or queue the observation for the **SME Conversational Flow**.

4 Quality Observability Architecture

To pipe Quality telemetry into the DPI-LS Engine, enterprises utilize external LLM-as-a-judge platforms (like LangSmith or TruEra). However, calculating Accuracy and Hallucination requires deep integration into the **RAG (Retrieval-Augmented Generation)** pipeline.

4.1 RAG Observability & Context Entailment

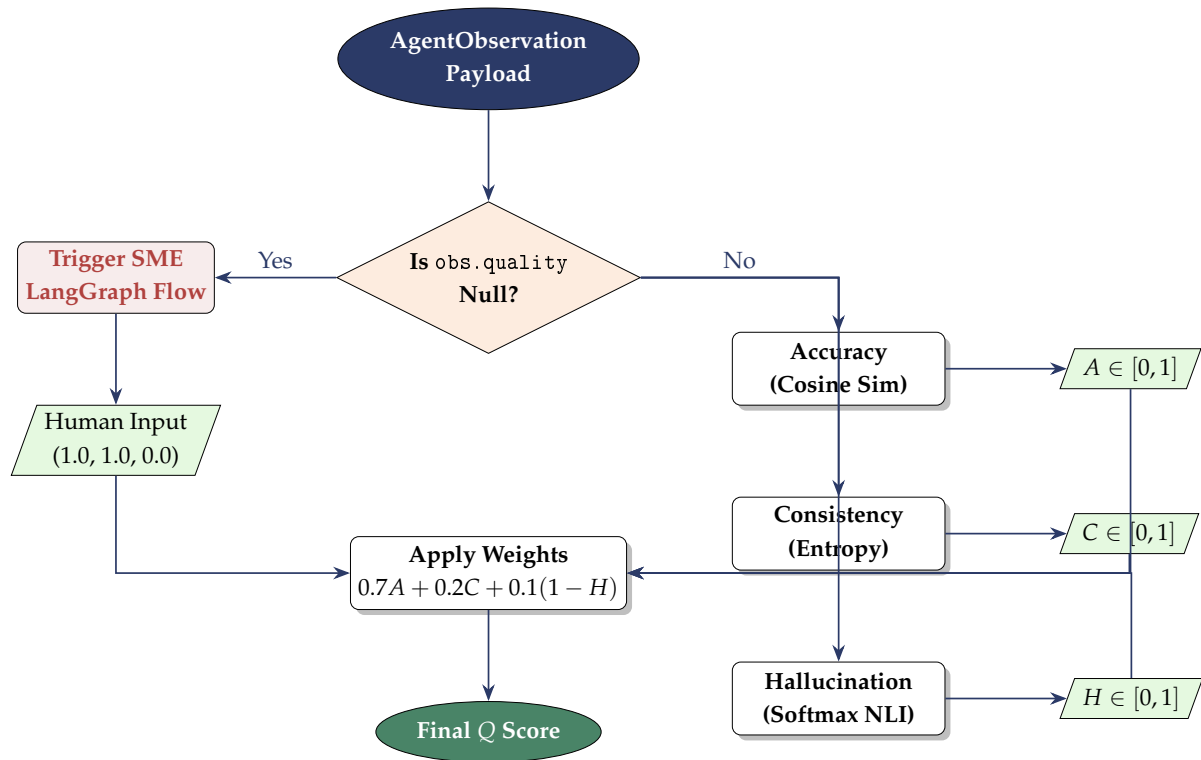
The agent cannot be judged in a vacuum. To measure if an agent hallucinated, the LLM-Judge must have access to both the *Prompt* and the *Retrieved Context* from the Vector Database.



As shown above, the LangSmith LLM Judge intercepts both the raw output from the agent AND the underlying vector chunks retrieved from Pinecone. By comparing the two, it calculates the Hallucination Rate using the Context Entailment mathematics detailed in Section 3.2.

4.2 The Quality Score Calculation Flow

The diagram below illustrates the exact programmatic flow of data from an Agent Observation through the DPI-LS Engine to generate the final Quality metric.



4.3 The SME Fallback Workflow

When the LLM Judge encounters an ambiguous output (resulting in a `None` payload), the DPI-LS Engine immediately suspends the automated scoring process. It queues the observation into a LangGraph Human-in-the-Loop workflow. An SME reviews the query, context, and output, manually grading the 0-1 floats.

5 Business Scenarios & Root Cause Analysis

The following scenarios demonstrate how the Quality dimension reacts to real-world AI incidents.

Scenario 1: The Hallucination Spike

! 1. The Event (Fact Fabrication): A customer service agent uses an outdated embedding model. While answering a user query about a refund policy, it hallucinates a "100% No-Questions-Asked" guarantee that does not exist.

Q 2. Observability Impact: LangSmith's RAG-Evaluation pipeline compares the agent's output against the ground-truth vector DB. It detects the fabrication. The telemetry payload reflects an Accuracy of 0.40 and a Hallucination Rate of 0.90.

📊 3. Mathematical Resolution:

- $Q = (0.7 \times 0.40) + (0.2 \times 0.95) + (0.1 \times (1 - 0.90))$
- $Q = 0.28 + 0.19 + 0.01 = \mathbf{0.48}$ (48%)

The low Q score tanks the agent's overall rating, alerting the ML Ops team to immediately update the RAG chunking strategy.

Scenario 2: The SME Override

📄 1. The Event (Complex Output): A legal-review agent generates a 50-page analysis of a merger contract. TruLens attempts to score it, but times out due to context window limits, returning None for Quality.

👥 2. Human-in-the-Loop Impact: The DPI-LS Engine receives the payload. Detecting the None value, it pauses the engine and pushes a notification to a Senior Legal SME via a Slack/LangGraph integration.

🏆 3. Mathematical Resolution: The SME reviews the document manually, determines it is flawless, and clicks "Approve". The engine manually sets Accuracy to 1.0, Consistency to 1.0, and Hallucination to 0.0, driving the Q score to a perfect 1.0 and allowing the Execution pipeline to resume.

Scenario 3: Prompt Drift & Inconsistency

✂️ **1. The Event (Stochastic Output):** A high-temperature agent is queried 5 times about the same data. It returns the correct numbers every time (Accuracy = 1.0) and invents nothing (Hallucination = 0.0). However, the formatting and phrasing are wildly different each time.

🔍 **2. Observability Impact:** Self-Consistency checks detect massive entropy. The Consistency score plummets to 0.40.

📊 **3. Mathematical Resolution:**

- $Q = (0.7 \times 1.0) + (0.2 \times 0.40) + (0.1 \times (1 - 0.0))$
- $Q = 0.70 + 0.08 + 0.10 = \mathbf{0.88}$ (88%)

Because Consistency only carries a 20% weight, the agent survives the incident and stays in the "Excellent" band, but ML Ops is notified to lower the model's temperature parameter.

Scenario 4: The Perfect Autonomous Run

📋 **1. The Event (Fully Optimized Run):** An internal HR agent uses few-shot prompting, dynamic RAG chunking, and a low temperature.

📊 **2. Mathematical Resolution:** LangSmith detects perfect alignment with the Pinecone Vector DB, generating $A = 0.99$, $C = 1.0$, and $H = 0.0$.

- $Q = (0.7 \times 0.99) + (0.2 \times 1.0) + (0.1 \times (1 - 0.0))$
- $Q = 0.693 + 0.20 + 0.10 = \mathbf{0.993}$ (99.3%)

The agent easily clears the 0.85 threshold, running completely autonomously without human intervention.

6 Telemetry & Integrations

Quality data is never localized. An enterprise agent must be evaluated by external LLM judges and observability platforms to prevent grading bias. To calculate an accurate Quality score, the DPI-LS ingestion adapters must pull evaluation telemetry from diverse ML Ops sources.

The table below maps how ML Ops systems translate raw evaluation data into formalized DPI-LS Quality metrics.

Quality Data	What It Measures	Source System	Actual Data Comes From
Semantic Accuracy	Cosine similarity vs ground truth	LangSmith Evaluation	LangSmith API
Context Entailment	Hallucination detection in RAG	TruEra / TruLens	TruLens UI SDK
Self-Consistency	Output stability over k iterations	AgentOps	AgentOps Dashboard
Human Feedback (RLHF)	SME thumbs up / down	LangSmith Annotation	LangSmith Webhook
Fact-Checking	External fact verification	Factool / Ragas	Ragas Pipeline
Prompt Injection	Jailbreak attempts detection	Lakera Guard	Lakera API
Toxicity & Bias	Unsafe output generation	Nvidia NeMo Guardrails	NeMo Log Export
Context Relevance	Did the RAG fetch correct data?	Arize Phoenix	Arize Traces
JSON Schema Compliance	Structural output validation	Pydantic / BAML	Python Validator Output
PII / Data Leakage	Sensitive data detection	Microsoft Presidio	Presidio Analyzer
Answer Relevance	Alignment with user intent	OpenAI Evals	OpenAI SDK
Context Recall	Completeness of retrieved facts	LlamaIndex Evaluator	LlamaIndex Logs
Custom Domain Rules	Regex / Keyword matching	Internal Python Judges	Custom Metric DB

7 Quality Telemetry Contract

The DPI-LS framework relies on a strictly typed Python pydantic contract (`contract/models.py`) to pass telemetry from LangSmith/TruEra into the Engine. Below is the mapping for the Quality block.

Contract Field	Python Type	Bounds	Description & Fallback Rule
quality	Optional[Quality]	Nullable	The root block. If None, engine skips math and queues SME flow.
accuracy	float	[0.0, 1.0]	Semantic Cosine Similarity against ground truth. Hard-clipped if > 1.0 .
consistency	float	[0.0, 1.0]	Self-consistency entropy map.
hallucination_rate	float	[0.0, 1.0]	The ratio of sentences tagged as Contradiction. Note: Engine inverts this $(1 - H)$.

8 Quality Thresholds & Alerting

To standardize autonomous agent operations, the DPI-LS framework establishes strict Quality band thresholds. These dictate when an agent can run fully autonomously versus when the infrastructure Circuit Breaker must intervene.

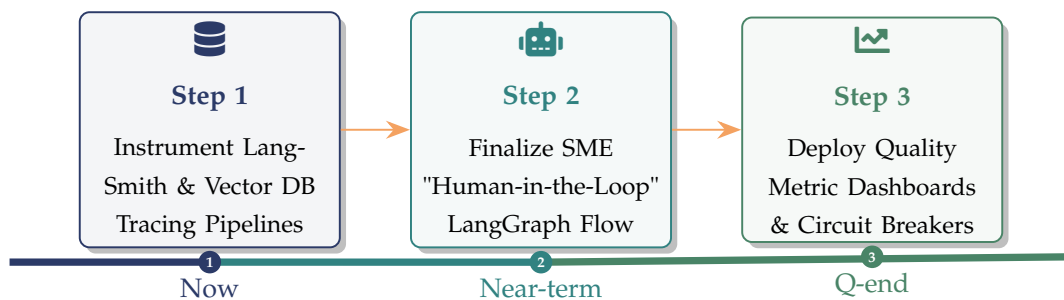
Score Range	Band Classification	Infrastructure Action
$Q \geq 0.85$	Excellent	✅ Agent operates autonomously. Output is trusted immediately.
$0.60 \leq Q < 0.85$	Acceptable	⚠️ Output is flagged for async human review. Operations continue but are closely monitored.
$Q < 0.60$	Critical Failure	❌ Circuit Breaker Triggered. Agent execution is halted. SME required.

9 Conclusion & Next Steps

The Quality Dimension Requires Empirical, Ground-Truth Validation

The Quality dimension ensures that an agent is not merely fast and cheap, but fundamentally reliable. By mathematically rigorously combining Accuracy, Consistency, and Hallucination tracking with robust RAG observability and human-in-the-loop SME overrides, the DPI-LS framework guarantees that enterprise AI applications can scale autonomously without risking brand damage or operational integrity.

9.1 Next Steps Roadmap



Document Developed By
AI Team
Malkapurapu Sivamanikanta

Digital Performance Index — Quality Dimension (Q) — Version 2.5

Architectural Reference • Confidential